PROJECT REPORT

SearchIQ Typeahead System

Professional submission report for a locally reproducible search typeahead system. It consolidates architecture, deterministic dataset loading, APIs, design decisions, trade-offs, performance evidence, screenshots, and validation results.

Runtime	Primary data size	Source of truth	Cache nodes
Node.js 24+	100,000 queries	SQLite	3 logical nodes

1. Executive Summary

SearchIQ is a locally runnable autocomplete system that returns top prefix suggestions, supports popularity-based and recency-aware ranking, uses a cache-before-database read path, and reduces write pressure through batched query-count updates.

The implementation is deliberately evaluation-friendly: it runs without external services, produces a deterministic 100,000-row dataset, and documents where a production version would use Redis Cluster, a scalable datastore, and a durable queue.

2. Architecture Overview

The system has four layers: React + Vite frontend, Express API, SQLite source of truth, and a simulated distributed cache implemented as three logical Map-based cache nodes routed through a consistent-hash ring.

```
User Browser (React + Vite UI)
| GET /suggest, POST /search, GET /cache/debug, GET /trending, GET /metrics
v
Express API (createApp.js)
+--> SuggestionService --> SQLite queries table
+--> TrendingService --> rolling one-hour activity window
+--> BatchWriter --> aggregated SQLite flushes
+--> ConsistentHashRing --> cache-node-1 / cache-node-2 / cache-node-3
```

Suggestion request flow

Step	Behavior
1	The UI waits for a 280ms debounce, then calls GET /suggest?q=<prefix>&ranking=<mode>.
2	The API normalizes the prefix and builds a ranking-specific cache key such as basic:iph or trending:iph.
3	The consistent-hash ring selects the logical cache node that owns the key.
4	On cache hit, suggestions are returned from cache; on miss, SQLite is queried and the result is cached.
5	POST /search updates recent activity immediately, invalidates affected prefixes, and coalesces DB writes in the batch buffer.

3. Dataset Source and Loading Instructions

No external dataset download is required. The project deterministically generates a CSV in query,count format using src/scripts/generateDataset.js and ingests it into SQLite using src/scripts/ingestDataset.js. This makes the submission reproducible for evaluation while still meeting the 100,000-row requirement.

Command	Purpose
npm run seed:small	Generate and ingest a 5,000-row development dataset for quick local checks.
npm run seed	Generate and ingest the full 100,000-row dataset used for final validation.

4. API Documentation

Method	Route	Purpose	Key behavior
GET	/suggest?q=<prefix>	Return up to 10 prefix suggestions.	Case-insensitive prefix match; basic ranking sorts by count DESC; cache checked before SQLite.
GET	/suggest?q=<prefix>&ranking=trending	Return recency-aware suggestions.	Uses score = allTimeCount + recentCountLastHour * 50.
POST	/search	Record a search submission.	Returns { "message": "Searched" }, updates recent activity, invalidates prefixes, and buffers DB update.
GET	/cache/debug?prefix=<prefix>	Inspect cache routing and TTL.	Shows normalized key, assigned cache node, hit/miss/expired status, and TTL.
GET	/trending?limit=<n>	Return current trending list.	Used by the UI signals panel.
GET	/metrics	Return system metrics.	Reports cache hit rate, DB operation counters, and write-reduction evidence.

Representative API responses

```
GET /suggest?q=iph
{
"query": "iph",
"ranking": "basic",
"source": "database",
"cacheStatus": "miss",
"cacheNode": "cache-node-3",
"suggestions": [{ "query": "iphone 15 pro max price", "count": 950000 }]
}

POST /search
{ "message": "Searched" }
```

5. Design Choices and Trade-offs

Design area	Chosen implementation	Why it was chosen	Trade-off / production alternative
Data store	SQLite via node:sqlite	Simple, deterministic, and easy to run locally.	Not horizontally scalable; production alternative: PostgreSQL, DynamoDB, or Cassandra.
Cache layer	Three logical Map-based cache nodes	Demonstrates cache ownership, prefix caching, TTL, and invalidation without external services.	Local simulation only; production alternative: Redis Cluster or Memcached.
Partitioning	Consistent hash ring with virtual nodes	Shows predictable distribution of prefix keys across cache owners.	Not a full distributed cache cluster.
Trending	Rolling one-hour activity window	Recent searches can reshape ranking while all-time popularity remains important.	Not personalized or ML-driven.
Write path	In-memory batch writer	Coalesces repeated submissions and reduces write amplification.	Pending increments can be lost on crash; production alternative: Kafka, Redis Streams, SQS, or DB-backed queue.

6. Performance Report

The measurements below were collected locally from the current implementation. They are included to prove that dataset loading, cache-aside reads, and write coalescing are not only described but also measured.

Seed performance

Command	Rows inserted	CSV generation	Ingestion	Total duration
npm run seed:small	5,000	55ms	870ms	975ms
npm run seed	100,000	546ms	4323ms	4879ms

Suggestion benchmark

Measurement	Measured value
Cold /suggest?q=iph	10.94ms wall-clock, 6.25ms API-reported latency
Warm /suggest?q=iph p95 over 25 requests	3.28ms wall-clock, 0.22ms API-reported latency
Warm /suggest?q=iph&ranking=trending p95 over 25 requests	2.05ms wall-clock, 0.18ms API-reported latency
Cache hit rate after warm-up	96%

Batch-write evidence

Metric	Observed value
Submissions / flushes	16 submissions, 6 flushes
Pending entries / searches	1 pending entry, 4 pending searches
Writes avoided / reduction	10 writes avoided, 63% reduction

7. Requirement-to-Implementation Mapping

Core functional requirements

Requirement	Implementation	Evidence / file
100,000 dataset in query,count format	Deterministic generator plus bulk SQLite ingestion.	src/scripts/generateDataset.js, src/scripts/ingestDataset.js, npm run seed
Search UI with suggestion dropdown	React module with 280ms debounce, ranking toggle, keyboard navigation, loading/error states, and inline request evidence.	src/App.jsx, src/styles/app.css
GET /suggest?q=<prefix>	Top 10 prefix suggestions; basic mode sorted by count DESC.	src/api/createApp.js, src/services/suggestionService.js, tests
POST /search	Returns { "message": "Searched" }, records recent activity, invalidates prefixes, and buffers DB updates.	src/services/searchService.js, src/services/batchWriter.js, tests
GET /cache/debug?prefix=<prefix>	Returns normalized prefix, assigned node, cache status, TTL, and node stats.	src/api/createApp.js, screenshots, tests
Trending searches	One-hour recent activity window and transparent score formula.	src/services/trendingService.js, src/services/suggestionService.js, tests
Batch writes	In-memory aggregation buffer, size/interval flush, and SQLite transaction writes.	src/services/batchWriter.js, src/db/database.js, tests

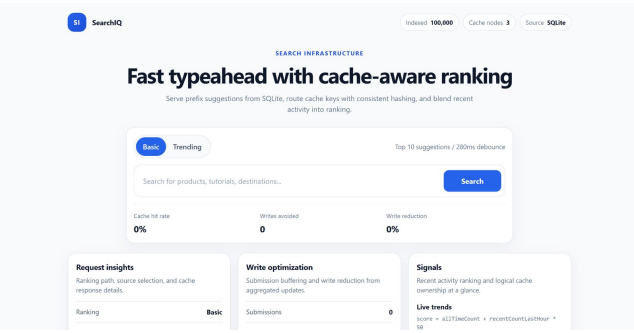
Design and evidence requirements

Requirement area	Implementation / explanation	Evidence / file
Cache-before-database flow	Suggestion service checks cache before SQLite and returns source/cacheStatus metadata.	src/services/suggestionService.js, docs/architecture.md
Prefix-level cache entries	Cache keys are scoped by ranking mode and prefix, e.g. basic:iph and trending:iph.	src/services/distributedCache.js
Multiple logical cache nodes	Three logical nodes with a consistent-hash ring and virtual nodes.	src/services/consistentHashRing.js, tests
Performance evidence	Seed timings, p95 latency, cache hit rate, and write-reduction metrics are documented.	docs/performance-report.md, src/scripts/benchmarkSuggest.js
Screenshot evidence	Actual UI screenshots are embedded in this PDF and retained under docs/screenshots/.	Section 8, docs/screenshots/README.md

8. Screenshot Evidence

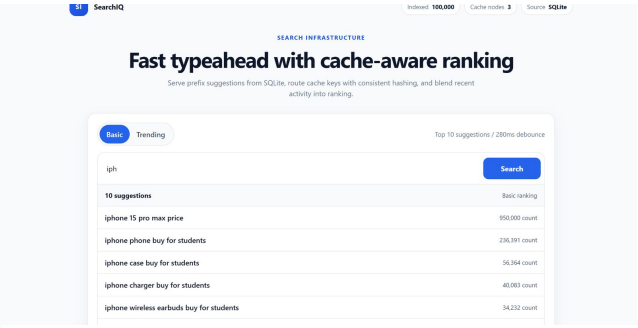
The screenshots below are embedded as compact evidence, not as a full-page dump. The original files should remain in the repository under docs/screenshots/.

home.png - initial UI



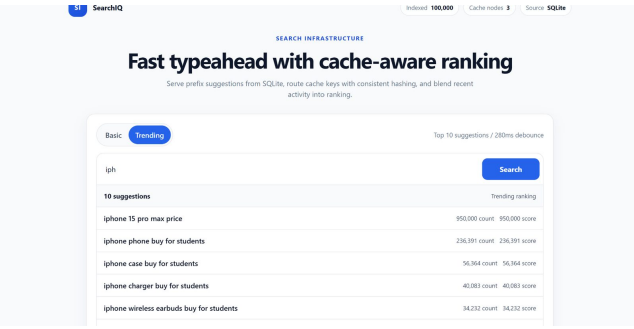
Search interface, 100,000 indexed records, SQLite source, and three logical cache nodes.

suggestions.png - prefix results



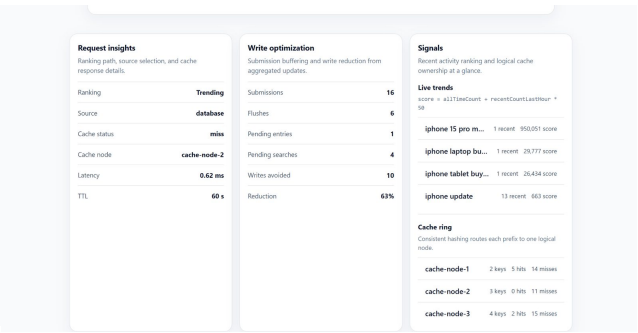
Basic ranking for prefix iph with top matching suggestions and counts.

trending.png - recency-aware mode



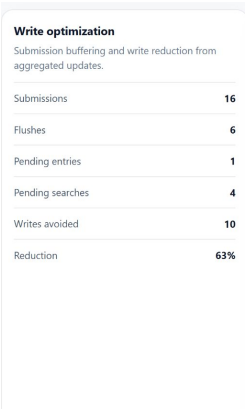
Trending ranking selected; scores combine all-time count and recent activity.

cache and batch evidence



Request insights, cache ownership, live trends, writes avoided, and write reduction.

batch-metrics.png - write coalescing evidence



Dedicated batch-write metrics: submissions, flushes, pending entries/searches, writes avoided, and 63% reduction.

9. Testing and Validation

The final validation sequence focuses on correctness and reproducibility. The optional benchmark requires the API server to be running; the recorded benchmark values are included in the performance section.

Command	Purpose	Latest result
npm run seed:small	Validate fast seed path and smaller reproducible dataset.	Pass
npm run seed	Validate full deterministic 100,000-row dataset load.	Pass
npm test	Verify API behavior, cache routing, trending ranking, and batch-write logic.	13/13 tests passed
npm run build	Verify frontend production build output.	Pass
npm run benchmark	Measure cold/warm suggestion latency and cache hit rate against running API.	Cold 10.94ms; warm p95 3.28ms; trending p95 2.05ms; hit rate 96%

10. Conclusion

This submission meets the project scope with a clear and reproducible implementation. It includes a 100,000-row deterministic dataset, a functional typeahead UI, cache-aside suggestion flow, consistent-hash cache ownership, trending ranking, batched persistence, screenshot evidence, tests, and measured local performance.

The architecture intentionally avoids unnecessary infrastructure for local evaluation while clearly documenting production alternatives such as Redis Cluster, a scalable datastore, a trie/search index, and a durable queue.